

Bölüm 4 — Sözdizimi ve Sözlüksel Analiz (Chapter 4)

Programlama Dilleri

Sözdizimi Analizi & Ayrıştırıcılar

Tüm Sorular — İngilizce & Türkçe + Durum Diyagramları & İz Tabloları

Kaynak Kitap:

Concepts of Programming Languages

Robert W. Sebesta — 12. Baskı

27

Gözden Geçirme
Sorusu

9

Problem
Seti Sorusu

12

Programlama
Alıştırması

Durum
Diagr.
+ İz
Tabloları

- Gözden Geçirme Soruları
- Problem Seti
- Programlama Alıştırmaları
- Durum Diyagramı / İz Tablosu içeren sorular

BÖLÜM 4 — GÖZDEN GEÇİRME SORULARI (Review Questions) • 27 Soru**Soru 1**

EN: What are three reasons why syntax analyzers are based on grammars?

TR: Sözdizimi analizörlerinin (syntax analyzers) gramerlere dayandırılmasının üç nedeni nedir?

● Cevap / Yönlendirme

(1) Tanım kesinliği: Gramer kuralları dilin sözdizimini tam ve belirsizliğe yer vermez biçimde tanımlar; bu sayede ayrıştırıcı (parser) hangi dizilerin geçerli olduğunu kesin olarak bilir. (2) Ayrıştırıcı üretimi: Gramerden otomatik olarak ayrıştırıcı üretilebilir (yacc, ANTLR gibi araçlar). (3) Okunabilirlik: Gramer tabanlı tanımlar doğal dil açıklamalarına kıyasla çok daha net ve tutarlıdır; uygulama ve belgeleme için doğrudan temel oluşturur.

Soru 2

EN: Explain the three reasons why lexical analysis is separated from syntax analysis.

TR: Sözlüksel analizin (lexical analysis) sözdizimi analizinden neden ayrıldığını açıklayan üç nedeni açıklayın.

● Cevap / Yönlendirme

(1) Tasarım basitliği: Sözdizimi analizörü daha küçük ve temiz bir gramerle çalışır; boşluk ve yorum gibi düşük düzey ayrıntılar sözdizimini karmaşıklaştırmaz. (2) Derleyici verimliliği: Sözlüksel analiz, düzenli dil tanıma için optimize edilmiş özel algoritmalar kullanır (DFA); bu aşama programın önemli bir bölümünü oluşturan karakter işlemeyi hızlandırır. (3) Taşınabilirlik: Giriş aygıtına özgü karakter seti gibi platform bağımlı ayrıntılar sözlüksel analizöre hapsedilir; sözdizimi analizörü platform bağımsız kalır.

Soru 3

EN: Define lexeme and token.

TR: Sözcük birimi (lexeme) ve belirteç (token) kavramlarını tanımlayın.

● Cevap / Yönlendirme

Sözcük birimi (lexeme), kaynak kodda gerçek olarak bulunan karakter dizisidir — örneğin `if`, `toplam`, `3.14`, `>=`. Belirteç (token) ise bir sözcük birimini sınıflandıran soyut sembol veya kategoridir — örneğin `ANAHTAR_KELIME`, `TANIMLAYICI`, `SABIT`, `OPERATOR`. Bir token genellikle `{token_tipi, değer}` çiftiyle temsil edilir. Sözlüksel analizör, kaynak kodun sözcük birimlerini tarayarak bunları token'lara dönüştürür.

Soru 4

EN: What are the primary tasks of a lexical analyzer?

TR: Sözlüksel analizörün (lexical analyzer / scanner) birincil görevleri nelerdir?

● **Cevap / Yönlendirme**

Sözlüksel analizörün başlıca görevleri: (1) Kaynak koddan karakterleri tek tek okumak, (2) Karakterleri sözcük birimlerine (lexemes) gruplandırmak, (3) Her sözcük birimini uygun token tipine sınıflandırmak, (4) Boşluk ve yorum satırlarını atlayarak görmezden gelmek, (5) Sayı ve dize sabitlerinin değerlerini hesaplamak/dönüştürmek, (6) Üretilen token'ları sözdizimi analizörüne iletmek.

Soru 5

EN: Describe briefly the three approaches to building a lexical analyzer.

TR: Sözlüksel analizör oluşturma'nın üç yaklaşımını kısaca açıklayın.

● **Cevap / Yönlendirme**

(1) Otomata teorisinden üretim (Lexer üretici kullanma): lex/flex gibi araçlar, düzenli ifade (regular expression) tanımlarından otomatik olarak sözlüksel analizör üretir — hızlı ve güvenilir. (2) Durum geçiş diyagramı ile elle kodlama: Programcı token tanıma mantığını bir durum makinesi (state machine) olarak elle kodlar — kontrol ve verimlilik sağlar. (3) Genel amaçlı bir programlama dilinde doğrudan uygulama: if-else yapılarıyla karakter karşılaştırmaları kullanılır — esnek ama daha zahmetli.

Soru 6

EN: What is a state transition diagram?

TR: Durum geçiş diyagramı (state transition diagram) nedir?

● **Cevap / Yönlendirme**

Durum geçiş diyagramı (state transition diagram), bir sonlu durum otomatının (finite automaton) görsel gösterimidir. Düğümler (daireler) durumları, yönlü kenarlar (oklar) geçişleri temsil eder; her geçiş üzerinde o geçişi tetikleyen giriş sembolü yazılır. Kabul eden durum (accepting state) çift daire ile gösterilir. Sözlüksel analizör tasarımında her token tipi için ayrı bir diyagram ya da birleşik bir diyagram kullanılır.

Soru 7

EN: Why are character classes used, rather than individual characters, for the letter and digit transitions of a state diagram for a lexical analyzer?

TR: Sözlüksel analizör için durum diyagramında harf ve rakam geçişlerinde tek tek karakterler yerine neden karakter sınıfları (character classes) kullanılır?

● **Cevap / Yönlendirme**

Karakter sınıfları kullanılmasının temel nedeni, diyagramı yönetilebilir boyutta tutmaktır. Bireysel karakterler kullanılsaydı, 26 harf için 26 ayrı geçiş oku gerekecekti; bu hem diyagramı aşırı karmaşık hem de kodu gereksiz uzun yapardı. 'letter' (harf) ve 'digit' (rakam) gibi sınıflar, ilgili tüm karakterleri tek bir geçişle temsil eder; bu durum diyagramın okunabilirliğini ve kodun bakımını kolaylaştırır.

Soru 8

EN: What are the two distinct goals of syntax analysis?

TR: Sözdizimi analizinin (syntax analysis) iki farklı amacı nedir?

● Cevap / Yönlendirme

(1) Tanıma (Recognition): Giriş programının kaynak dilin gramerine uygun olup olmadığını doğrular; sözdizimi hataları varsa bildirir. (2) Yapı çıkarımı (Structure extraction): Giriş programının hiyerarşik yapısını ortaya çıkaran ayrıştırma ağacını (parse tree) ya da soyut sözdizimi ağacını (abstract syntax tree — AST) oluşturur. Bu ağaç daha sonra anlambilim analizi ve kod üretimi için kullanılır.

Soru 9

EN: Describe the differences between top-down and bottom-up parsers.

TR: Yukarıdan aşağıya (top-down) ve aşağıdan yukarıya (bottom-up) ayrıştırıcılar arasındaki farkları açıklayın.

● Cevap / Yönlendirme

Yukarıdan aşağıya ayrıştırıcı (top-down parser): Ayrıştırmaya başlangıç sembolünden (start symbol) başlar ve üretim kurallarını ileriye doğru uygulayarak girdi dizisini türetmeye çalışır. Recursive descent ve LL parsers bu gruba girer. Aşağıdan yukarıya ayrıştırıcı (bottom-up parser): Giriş dizisinden başlar ve başlangıç sembolüne ulaşana kadar üretim kurallarını geriye doğru (sağdan sola) uygular. LR parsers (SLR, LALR, Canonical LR) bu gruba girer. Bottom-up ayrıştırıcılar daha geniş bir gramer sınıfını destekler.

Soru 10

EN: Describe the parsing problem for a top-down parser.

TR: Yukarıdan aşağıya ayrıştırıcı (top-down parser) için ayrıştırma problemini açıklayın.

● Cevap / Yönlendirme

Yukarıdan aşağıya ayrıştırıcının temel problemi, bir non-terminal için hangi üretim kuralının uygulanacağına karar vermektir. Birden fazla üretim kuralı varsa ve yanlış kural seçilirse geri izleme (backtracking) gerekirken bu da verimsizliğe yol açar. Pratik LL(k) ayrıştırıcılar geri izlemeden kaçınmak için k adet ileriye bakış (lookahead) sembolü kullanır. Sol özyinelemeli (left-recursive) ve sola faktörleştirilemeyen gramerler top-down ayrıştırma için sorun oluşturur.

Soru 11

EN: Describe the parsing problem for a bottom-up parser.

TR: Aşağıdan yukarıya ayrıştırıcı (bottom-up parser) için ayrıştırma problemini açıklayın.

● Cevap / Yönlendirme

Aşağıdan yukarıya ayrıştırıcının temel problemi iki eylem arasında karar vermektir: (1) Kaydırma (Shift): Sonraki giriş sembolünü yığına (stack) itmek, (2) İndirgeme (Reduce): Yığının tepesindeki sembolleri bir üretim kuralının sağ tarafıyla eşleştirip sol tarafıyla değiştirmek. Bu shift-reduce kararı, mevcut yığın içeriğine ve ileriye bakış (lookahead) sembolüne göre belirlenir. LR ayrıştırma tablosu bu kararı hızlı ve belirsizlik olmadan almayı sağlar.

Soru 12

EN: Explain why compilers use parsing algorithms that work on only a subset of all grammars.

TR: Derleyicilerin neden tüm gramerlerin yalnızca bir alt kümesi üzerinde çalışan ayrıştırma algoritmalarını kullandığını açıklayın.

● Cevap / Yönlendirme

Genel bağlamdan bağımsız gramerler için ayrıştırma, $O(n^3)$ zaman karmaşıklığına sahiptir (CYK algoritması gibi) — bu büyük programlar için kabul edilemez yavaştır. Derleyiciler, belirli kısıtlamalara (sol özyineleme yok, belirsizlik yok, belirli FIRST/FOLLOW kümesi özellikleri) uyan gramer alt kümeleri için $O(n)$ doğrusal zamanda çalışan algoritmaları (LL(1), LALR(1)) kullanır. Pratikte programlama dilleri zaten bu kısıtlamalara uyacak biçimde tasarlanır.

Soru 13

EN: Why are named constants used, rather than numbers, for token codes?

TR: Token kodları için sayılar yerine neden adlandırılmış sabitler (named constants) kullanılır?

● Cevap / Yönlendirme

Adlandırılmış sabitler (named constants) kullanılmasının iki temel nedeni: (1) Okunabilirlik: INT_CODE veya IF_CODE gibi isimler kodun anlamını açıkça ifade eder; 35 veya 31 gibi sayısal kodlar ise sezgisel değildir ve hata yapmayı kolaylaştırır. (2) Bakım kolaylığı: Token kodu değiştirilmesi gerektiğinde yalnızca tek bir yerdeki sabit tanımı güncellenir; sayısal kod kullanılmış olsaydı tüm dosyalarda o sayıyı aramak gerekirdi. Bu OOP'deki magic number'dan kaçınma ilkesiyle aynı mantığı taşır.

Soru 14

EN: Describe how a recursive-descent parsing subprogram is written for a rule with a single RHS.

TR: Tek bir sağ tarafı (single RHS) olan bir kural için özyinelemeli iniş (recursive-descent) ayrıştırma alt programının nasıl yazıldığını açıklayın.

● Cevap / Yönlendirme

Tek RHS'li bir kural için alt program yapısı:

(1) Kural: $\langle A \rangle \rightarrow aBC$

(2) Alt program: `void A() {`

`if (nextToken == 'a') match('a');`

`else error();`

`B(); // B non-terminalini çağır`

`C(); // C non-terminalini çağır`

`}`

Kuralın sağ tarafındaki her sembol için:

- Terminal sembol: `match()` ile karşılaştır ve tüket
- Non-terminal sembol: O non-terminal'ın alt programını çağır

Soru 15

EN: Explain the two grammar characteristics that prohibit them from being used as the basis for a top-down parser.

TR: Gramerlerin yukarıdan aşağıya ayrıştırıcı için kullanılmasını engelleyen iki gramer özelliğini açıklayın.

● Cevap / Yönlendirme

(1) Sol özyineleme (Left recursion): $\langle A \rangle \rightarrow \langle A \rangle \alpha$ gibi bir kural, yukarıdan aşağıya ayrıştırıcının sonsuz döngüye (infinite loop) girmesine yol açar; çünkü A'yı ayrıştırmak için önce A çağrılır. (2) Sola faktörleştirilemeyen kurallar (Non-left-factored rules): $\langle A \rangle \rightarrow \alpha\beta \mid \alpha\gamma$ gibi ortak ön eki olan iki kural varsa, hangi kuralın seçileceğine yalnızca α 'yı görerek karar verilemez; bu durum belirsizliğe ve geri izlemeye yol açar.

Soru 16

EN: What is the FIRST set for a given grammar and sentential form?

TR: Belirli bir gramer ve cümlesel form için FIRST kümesi nedir?

● Cevap / Yönlendirme

FIRST(α) kümesi, α cümlesel formundan türetililecek dizilerin başlayabileceği terminal sembollerin kümesidir. Formal olarak: $FIRST(\alpha) = \{a \mid \alpha \Rightarrow^* a\beta, a \text{ terminal}\}$. Eğer α boş dizeyi (ϵ) türetebiliyorsa FIRST(α) içine ϵ de eklenir. FIRST kümeleri, LL(1) ayrıştırıcılarda hangi üretim kuralının seçileceğine karar vermek için kullanılır: eğer sonraki token FIRST(α)'daysak α 'yı türeten kuralı seçeriz.

Soru 17

EN: Describe the pairwise disjointness test.

TR: İkili ayrışıklık testini (pairwise disjointness test) açıklayın.

● Cevap / Yönlendirme

İkili ayrışıklık testi, bir gramerin LL(1) ayrıştırıcı için uygun olup olmadığını kontrol eder. Bir $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ kuralı için test: Her α_i ve α_j çifti ($i \neq j$) için $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset$ olmalıdır. Yani hiçbir iki alternatifi FIRST kümesi ortak terminal içermemelidir. Bu koşul sağlandığında, bir sonraki giriş tokena bakarak hangi kuralın seçileceği kesin biçimde belirlenir ve geri izleme gerekmez.

Soru 18

EN: *What is left factoring?*

TR: Sol faktörleştirme (left factoring) nedir?

● Cevap / Yönlendirme

Sol faktörleştirme (left factoring), top-down ayrıştırmayı engelleyen ortak ön ek sorununu gidermek için uygulanan bir gramer dönüşümü tekniğidir. Ortak ön eki α olan kurallar:

$\langle A \rangle \rightarrow \alpha\beta \mid \alpha\gamma$

şu biçime dönüştürülür:

$\langle A \rangle \rightarrow \alpha\langle A' \rangle$

$\langle A' \rangle \rightarrow \beta \mid \gamma$

Bu dönüşüm sayesinde α görüldüğünde $\langle A' \rangle$ alt programı çağrılır ve β ile γ arasındaki seçim daha sonra yapılır; böylece belirsizlik ortadan kalkar.

Soru 19

EN: *What is a phrase of a sentential form?*

TR: Cümlesel formun bir deyimi (phrase) nedir?

● Cevap / Yönlendirme

Bir cümlesel formun deyimi (phrase), o cümlesel formun ayrıştırma ağacında belirli bir non-terminal düğümün altındaki tüm terminal (yaprak) sembollerin oluşturduğu alt dizedir. Daha formal olarak: β , $\langle A \rangle \rightarrow^* \beta$ ise β , $\langle A \rangle$ 'nın türettiği bir deyimdir. Kısacası bir deyim, herhangi bir non-terminal'in sıfır veya daha fazla adımda ürettiği terminal dizisidir.

Soru 20

EN: *What is a simple phrase of a sentential form?*

TR: Cümlesel formun basit deyimi (simple phrase) nedir?

● Cevap / Yönlendirme

Basit deyim (simple phrase), tam olarak bir türetme adımıyla elde edilen deyimdir. Yani β , $\langle A \rangle \rightarrow \beta$ kuralını kullanarak doğrudan türetilmişse β basit bir deyimdir. Başka bir ifadeyle: basit deyim, ayrıştırma ağacında bir non-terminal düğümün doğrudan çocuklarının oluşturduğu dizedir (ara non-terminal olmaksızın). Her basit deyim aynı zamanda bir deyimdir, ancak her deyim basit bir deyim değildir.

Soru 21

EN: What is the handle of a sentential form?

TR: Cümlesel formun tutamacı (handle) nedir?

● Cevap / Yönlendirme

Tutamak (handle), bir sağ cümlesel formun en solunda bulunan basit deyimidir. Bottom-up ayrıştırmada tutamak, indirgeme (reduce) işlemi uygulanacak olan kısımdır: yığının tepesindeki bu dizi, karşılık gelen üretim kuralının sol tarafıyla değiştirilerek indirgeme yapılır. Tutamakin doğru biçimde tanımlanması, shift-reduce çatışmalarını (shift-reduce conflicts) çözümlmek için kritik öneme sahiptir.

Soru 22

EN: What is the mathematical machine on which both top-down and bottom-up parsers are based?

TR: Hem yukarıdan aşağıya hem de aşağıdan yukarıya ayrıştırıcıların dayandığı matematiksel makine nedir?

● Cevap / Yönlendirme

Her iki ayrıştırıcı türü de aşağı inişli otomat (pushdown automaton — PDA) adı verilen matematiksel makinenin üzerine kuruludur. PDA, bir sonlu durum otomatının yığın (stack) belleğiyle genişletilmiş hâlidir. Yığın, ayrıştırıcının o ana kadar işlediği durumları ve sembolleri saklamasına olanak tanır. Bağlamdan bağımsız (context-free) dillerin tam olarak PDA'lar tarafından tanınabileceği teorik olarak kanıtlanmıştır.

Soru 23

EN: Describe three advantages of LR parsers.

TR: LR ayrıştırıcılarının üç avantajını açıklayın.

● Cevap / Yönlendirme

(1) Güç: LR ayrıştırıcılar, LL ayrıştırıcıların tanımlayamadığı çok daha geniş bir bağlamdan bağımsız gramer sınıfını (sol özyinelemeli gramerler dahil) işleyebilir. (2) Hata tespiti: Her türlü sözdizimi hatası mümkün olan en erken noktada tespit edilir; gereksiz karakter tüketimi gerçekleşmez. (3) Verimlilik: LR ayrıştırma, giriş uzunluğuyla doğrusal $O(n)$ zamanda çalışır ve geri izleme gerektirmez; bu da derleyiciler için pratik kullanıma uygundur.

Soru 24

EN: What was Knuth's insight in developing the LR parsing technique?

TR: Knuth'un LR ayrıştırma tekniğini geliştirirken temel içgörüsü (insight) neydi?

● Cevap / Yönlendirme

Donald Knuth'un temel içgörüsü şuydu: Bir sağ cümlesel formun tutamacı (handle) her zaman ayrıştırma yığınının (parse stack) tepesindedir ve yığın içeriği tutamacı benzersiz biçimde tanımlamak için yeterlidir. Knuth, yığının en tepesinde neler olduğuna ve bir sonraki giriş sembolüne bakarak shift ya da reduce kararının hatasız verilebileceğini kanıtladı. Bu içgörü, LR parse tablolarının önceden hesaplanmasını mümkün kıldı.

Soru 25

EN: Describe the purpose of the ACTION table of an LR parser.

TR: Bir LR ayrıştırıcısının ACTION tablosunun amacını açıklayın.

● Cevap / Yönlendirme

ACTION tablosu, LR ayrıştırıcısının her adımda ne yapacağını belirler. Giriş: mevcut durum (state) ve ileriye bakış tokeni (lookahead token). Çıkış dört olası eylemden biridir: (1) Shift(s): Sonraki tokeni yığına it ve s durumuna geç; (2) Reduce(r): r numaralı kuralla indir; (3) Accept: Ayrıştırma başarıyla tamamlandı; (4) Error: Sözdizimi hatası tespit edildi. Tablo derleme öncesinde hesaplanır ve ayrıştırma sürecinde $O(1)$ zamanda sorgulanır.

Soru 26

EN: Describe the purpose of the GOTO table of an LR parser.

TR: Bir LR ayrıştırıcısının GOTO tablosunun amacını açıklayın.

● Cevap / Yönlendirme

GOTO tablosu, bir indirgeme (reduce) işlemi gerçekleştirildikten sonra gidilmesi gereken yeni durumu belirler. Giriş: mevcut durum ve indirgeme sonucu elde edilen non-terminal sembol. Çıkış: Geçilecek yeni durum numarasıdır. GOTO tablosu, indirgeme işlemi sırasında yığından silinen durumların yerine yeni durumu yüklemek için kullanılır; böylece ayrıştırıcı doğru devam durumunu bulur.

Soru 27

EN: Is left recursion a problem for LR parsers?

TR: Sol özyineleme (left recursion) LR ayrıştırıcıları için bir sorun mudur?

● Cevap / Yönlendirme

Hayır, sol özyineleme LR ayrıştırıcıları için bir sorun değildir. Top-down ayrıştırıcılarda sol özyinelemeli kurallar sonsuz döngüye yol açarken, LR (bottom-up) ayrıştırıcılar bu kuralları sorunsuz işler. Aksine sol özyinelemeli gramerler bazen sağ özyinelemeli muadillerine kıyasla daha kompakt ve doğal olabilir; örneğin $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$ kuralı + operatörünün sol birleşimliliğini (left associativity) doğal biçimde ifade eder.

BÖLÜM 4 — PROBLEM SETİ (Problem Set) • 9 Soru**Problem 1**

EN: Perform the pairwise disjointness test for the following grammar rules. (a) $A \rightarrow aB \mid b \mid cBB$ (b) $B \rightarrow aB \mid bA \mid aBb$ (c) $C \rightarrow aaA \mid b \mid caB$

TR: Aşağıdaki gramer kuralları için ikili ayrışıklık testini uygulayın.

● Cevap / Yönlendirme

İkili ayrışıklık testi: Her kural çifti için FIRST kümelerinin kesişimi boş olmalıdır.

(a) $A \rightarrow aB \mid b \mid cBB$:

$FIRST(aB) = \{a\}$, $FIRST(b) = \{b\}$, $FIRST(cBB) = \{c\}$

$\{a\} \cap \{b\} = \emptyset$, $\{a\} \cap \{c\} = \emptyset$, $\{b\} \cap \{c\} = \emptyset$ ✓ PASSES

(b) $B \rightarrow aB \mid bA \mid aBb$:

$FIRST(aB) = \{a\}$, $FIRST(bA) = \{b\}$, $FIRST(aBb) = \{a\}$

$\{a\} \cap \{b\} = \emptyset$, $\{a\} \cap \{a\} = \{a\} \neq \emptyset$ ✗ FAILS (aB ve aBb çakışıyor)

(c) $C \rightarrow aaA \mid b \mid caB$:

$FIRST(aaA) = \{a\}$, $FIRST(b) = \{b\}$, $FIRST(caB) = \{c\}$

Tüm kesişimler boş ✓ PASSES

Problem 2

EN: Perform the pairwise disjointness test for the following grammar rules. (a) $S \rightarrow aSb \mid bAA$ (b) $A \rightarrow b\{aB\} \mid a$ (c) $B \rightarrow aB \mid a$

TR: Aşağıdaki gramer kuralları için ikili ayrışıklık testini uygulayın.

● Cevap / Yönlendirme

İkili ayrışıklık testi:

(a) $S \rightarrow aSb \mid bAA$:

$FIRST(aSb) = \{a\}$, $FIRST(bAA) = \{b\}$

$\{a\} \cap \{b\} = \emptyset$ ✓ PASSES

(b) $A \rightarrow b\{aB\} \mid a$ (EBNF: b ardından sıfır veya daha fazla aB):

$FIRST(b\{aB\}) = \{b\}$, $FIRST(a) = \{a\}$

$\{b\} \cap \{a\} = \emptyset$ ✓ PASSES

(c) $B \rightarrow aB \mid a$:

$FIRST(aB) = \{a\}$, $FIRST(a) = \{a\}$

$\{a\} \cap \{a\} = \{a\} \neq \emptyset$ ✗ FAILS (her iki kural da a ile başlıyor, belirsizlik var)

Problem 3 [İz Tablosu ↓]

EN: Show a trace of the recursive descent parser given in Section 4.4.1 for the string $a + b * c$.

TR: Bölüm 4.4.1'deki özyinelemeli iniş ayrıştırıcısının $a + b * c$ dizesi için izini (trace) gösterin.

● Cevap / Yönlendirme

Gramer (EBNF):

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ (+|-) \langle \text{term} \rangle \}$

$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \{ (*|/) \langle \text{factor} \rangle \}$

$\langle \text{factor} \rangle \rightarrow \text{id} \mid (\langle \text{expr} \rangle)$

Ayrıştırma izi tablosu aşağıda gösterilmiştir.

● Özyinelemeli İniş İzi — $a + b * c$

Eylem / Action	Kalan Giriş / Remaining Input
expr() çağrıldı	a + b * c
term() çağrıldı	a + b * c
factor() çağrıldı	a + b * c
id gördü, eşleştirdi: a	+ b * c
factor() döndü	+ b * c
* yok → term döndü	+ b * c
term() döndü	+ b * c
+ gördü, eşleştirdi	b * c
term() çağrıldı	b * c
factor() çağrıldı	b * c
id gördü, eşleştirdi: b	* c
factor() döndü	* c
* gördü, eşleştirdi	c
factor() çağrıldı	c
id gördü, eşleştirdi: c	\$
factor() döndü	\$
* yok → term döndü	\$
term() döndü	\$
+ yok → expr döndü	\$
BAŞARILI (SUCCESS)	

Problem 4 [İz Tablosu ↓]

EN: Show a trace of the recursive descent parser given in Section 4.4.1 for the string $a * (b + c)$.

TR: Bölüm 4.4.1'deki özyinelemeli iniş ayrıştırıcısının $a * (b + c)$ dizesi için izini (trace) gösterin.

● Cevap / Yönlendirme

Gramer (EBNF):

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ (+|-) \langle \text{term} \rangle \}$

$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \{ (*|/) \langle \text{factor} \rangle \}$

$\langle \text{factor} \rangle \rightarrow \text{id} \mid (\langle \text{expr} \rangle)$

Ayrıştırma izi tablosu aşağıda gösterilmiştir.

● Özyinelemeli İniş İzi — $a * (b + c)$

Eylem / Action	Kalan Giriş / Remaining Input
expr() çağrıldı	a * (b + c)
term() çağrıldı	a * (b + c)
factor(): id a eşleşti	* (b + c)
* gördü, eşleşti	(b + c)
factor() çağrıldı	(b + c)
(gördü, eşleşti	b + c)
expr() çağrıldı	b + c)
term() çağrıldı	b + c)
factor(): id b	+ c)
term() döndü	+ c)
+ gördü, eşleşti	c)
term() çağrıldı	c)
factor(): id c	)
term() döndü	)
expr() döndü	)
) gördü, eşleşti	\$
factor() döndü	\$
* yok → term döndü	\$
term() döndü	\$
+ yok → expr döndü	\$
BAŞARILI (SUCCESS)	

Problem 5 [Parse Tree ↓]

EN: Given the grammar $S \rightarrow aAb|bBA$ $A \rightarrow ab|aAB$ $B \rightarrow aB|b$ — draw a parse tree for: (a) aaAbb (b) bBab (c) aaAbBb. Show phrases, simple phrases, and handle.

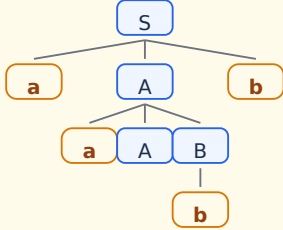
TR: Verilen gramer için parse tree çizin ve deyim, basit deyim ile tutamacı gösterin: (a) aaAbb (b) bBab (c) aaAbBb

● Cevap / Yönlendirme

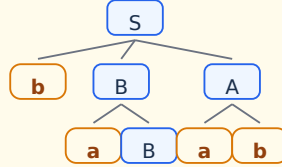
Ayrıştırma ağaçları ve analiz aşağıdaki görsel bölümde gösterilmiştir. Tanımlar:

- Deyim (Phrase): Herhangi bir non-terminal'in türettiği alt dizi
- Basit Deyim (Simple Phrase): Tek adımda türetilen deyim
- Tutamak (Handle): En soldaki basit deyim (indirgeme noktası)

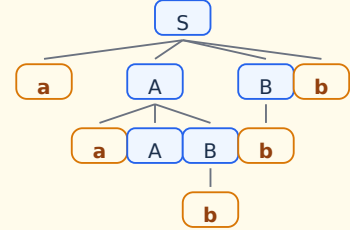
● 5a: aaAbb



● 5b: bBab



● 5c: aaAbBb



Problem 6 [Parse Tree ↓]

EN: Given the grammar $S \rightarrow AbB \mid bAc$ $A \rightarrow Ab \mid aBB$ $B \rightarrow Ac \mid cBb \mid c$ — draw a parse tree for: (a) aAcccbbc (b) AbcaBccb (c) baBcBbbc. Show phrases, simple phrases, handle.

TR: Verilen gramer için parse tree çizin, deyim/basit deyim/tutamak gösterin: (a) aAcccbbc (b) AbcaBccb (c) baBcBbbc

● Cevap / Yönlendirme

Bu problem, sağ cümlesel formların (right sentential forms) analizi için bottom-up ayrıştırma bilgisi gerektirir. Genel yaklaşım:

- (a) aAcccbbc: $B \rightarrow c$ ile en sağdaki B indirgenir $\rightarrow aAcccbB$ sonra $B \rightarrow cBb \rightarrow \dots$ S'ye ulaşana dek sürer.
 (b) AbcaBccb: $A \rightarrow Ab$, $B \rightarrow c$, $B \rightarrow cBb$ indirgeme adımlarıyla çözülür.
 (c) baBcBbbc: $S \rightarrow bAc \rightarrow b \cdot (Ab) \cdot c \rightarrow b \cdot (aBB) B \cdot c$ şeklinde geri iz sürülür.

Her sentential form için tutamak, en soldaki basit deyim (tek kuraldan türetilen yaprak dizisi) olarak belirlenir.

Problem 7 [LR İz Tablosu ↓]

EN: Show a complete parse for $id * (id + id)$ using the grammar and parse table in Section 4.5.3. Include stack, input, and action.

TR: Section 4.5.3'teki gramer ve ayrıştırma tablosunu kullanarak $id * (id + id)$ için tam ayrıştırma sürecini stack, input ve action sütunlarıyla gösterin.

● Cevap / Yönlendirme

LR ayrıştırma grameri (Section 4.5.3):

$E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid id$

Ayrıştırma izi aşağıdaki tabloda verilmiştir. Durum sayıları standart LALR(1) tablolarına dayanmaktadır.

● LR Ayrıştırma İzi — $id * (id + id)$

Yığın (Stack)	Giriş (Input)	Eylem (Action)
0	id*(id+id)\$	shift → goto 5
0 id 5	*(id+id)\$	reduce F→id
0 F 3	*(id+id)\$	reduce T→F
0 T 2	*(id+id)\$	shift * → goto 7
0 T 2 * 7	(id+id)\$	shift (→ goto 4
0 T 2 * 7 (4	id+id)\$	shift id → goto 5
0 T 2 * 7 (4 id 5	+id)\$	reduce F→id
0 T 2 * 7 (4 F 3	+id)\$	reduce T→F
0 T 2 * 7 (4 T 2	+id)\$	reduce E→T
0 T 2 * 7 (4 E 8	+id)\$	shift + → goto 6
0 T 2 * 7 (4 E 8 + 6	id)\$	shift id → goto 5
0 T 2 * 7 (4 E 8 + 6 id 5	)\$	reduce F→id
0 T 2 * 7 (4 E 8 + 6 F 3	)\$	reduce T→F
0 T 2 * 7 (4 E 8 + 6 T 9	)\$	reduce E→E+T
0 T 2 * 7 (4 E 8	)\$	shift) → goto 11
0 T 2 * 7 (4 E 8) 11	\$	reduce F→(E)
0 T 2 * 7 F 10	\$	reduce T→T*F
0 T 2	\$	reduce E→T
0 E 1	\$	ACCEPT

Problem 8 [LR İz Tablosu ↓]

EN: Show a complete parse for $(id + id) * id$ using the grammar and parse table in Section 4.5.3.

TR: Section 4.5.3'teki gramer ve ayrıştırma tablosunu kullanarak $(id + id) * id$ için tam ayrıştırma sürecini gösterin.

● Cevap / Yönlendirme

LR ayrıştırma grameri:

$E \rightarrow E+T \mid T, T \rightarrow T*F \mid F, F \rightarrow (E) \mid id$

Ayrıştırma izi aşağıdaki tabloda verilmiştir.

● LR Ayrıştırma İzi — $(id + id) * id$

Yığın (Stack)	Giriş (Input)	Eylem (Action)
0	(id+id)*id\$	shift (→ goto 4
0 (4	id+id)*id\$	shift id → goto 5
0 (4 id 5	+id)*id\$	reduce F→id
0 (4 F 3	+id)*id\$	reduce T→F
0 (4 T 2	+id)*id\$	reduce E→T
0 (4 E 8	+id)*id\$	shift + → goto 6
0 (4 E 8 + 6	id)*id\$	shift id → goto 5
0 (4 E 8 + 6 id 5	)*id\$	reduce F→id
0 (4 E 8 + 6 F 3	)*id\$	reduce T→F
0 (4 E 8 + 6 T 9	)*id\$	reduce E→E+T
0 (4 E 8	)*id\$	shift) → goto 11
0 (4 E 8) 11	*id\$	reduce F→(E)
0 F 3	*id\$	reduce T→F
0 T 2	*id\$	shift * → goto 7
0 T 2 * 7	id\$	shift id → goto 5
0 T 2 * 7 id 5	\$	reduce F→id
0 T 2 * 7 F 10	\$	reduce T→T*F
0 T 2	\$	reduce E→T
0 E 1	\$	ACCEPT

Problem 9

EN: Remove all left recursion from: $S \rightarrow Aa|Bb$ $A \rightarrow Aa|Abc|c|Sb$ $B \rightarrow bb$

TR: Aşağıdaki gramerden tüm sol özyinelemeyi kaldırın: $S \rightarrow Aa|Bb$ $A \rightarrow Aa|Abc|c|Sb$ $B \rightarrow bb$

● Cevap / Yönlendirme

Aho vd. algoritması doğrudan ve dolaylı sol özyinelemeyi kaldırır:

Adım 1: $B \rightarrow bb$ (sol özyineleme yok, değişmez)

Adım 2: $A \rightarrow Aa|Abc|c|Sb$ — Sb'deki S'yi genişlet:

$S \rightarrow Aa|Bb \rightarrow A \rightarrow Aa|Abc|c|Aab|Bbb$

A'daki doğrudan sol özyinelemeler (Aa, Abc, Aab) kaldırılır:

$A \rightarrow cA' | BbbA'$

$A' \rightarrow aA' | bcA' | abA' | \epsilon$

Adım 3: $S \rightarrow Aa|Bb$:

$S \rightarrow Aa | Bb$ (A ve B artık sol özyinelemesiz)

Sonuç: $S \rightarrow Aa|Bb$ $A \rightarrow cA'|BbbA'$ $A' \rightarrow aA'|bcA'|abA'|\epsilon$ $B \rightarrow bb$

BÖLÜM 4 — PROGRAMLAMA ALIŞTIRMALARI (Programming Exercises) • 12 Alıştırma

Alıştırma 1 [Durum Diyagramı ↓]

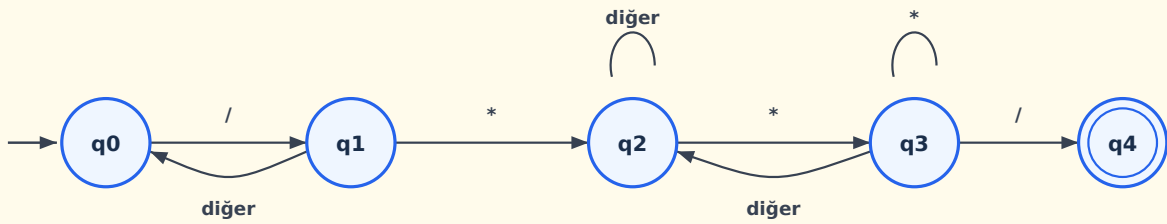
EN: Design a state diagram to recognize C-style comments that begin with `/*` and end with `*/`.

TR: `/*` ile başlayıp `*/` ile biten C-tarzı yorumları (comments) tanıyan bir durum diyagramı tasarlayın.

● Cevap / Yönlendirme

5 durumlu bir sonlu otomat gereklidir. q_0 : başlangıç, q_1 : `/` görüldü, q_2 : `/*` içinde (yorum gövdesi), q_3 : `/*` içinde `*` görüldü (olası kapanış), q_4 : `*/` görüldü (kabul — yorum tamamlandı). Durum diyagramı aşağıda gösterilmiştir.

○ Durum Diyagramı — C Tarzı `/* ... */` Yorumlar



q0	Başlangıç durumu
q1	<code>/</code> görüldü — <code>*</code> bekleniyor
q2	<code>/*</code> içinde — yorum gövdesi
q3	<code>/*</code> içinde <code>*</code> görüldü — olası kapanış
q4	KABUL — <code>*/</code> tamamlandı

Alıştırma 2 [Durum Diyagramı ↓]

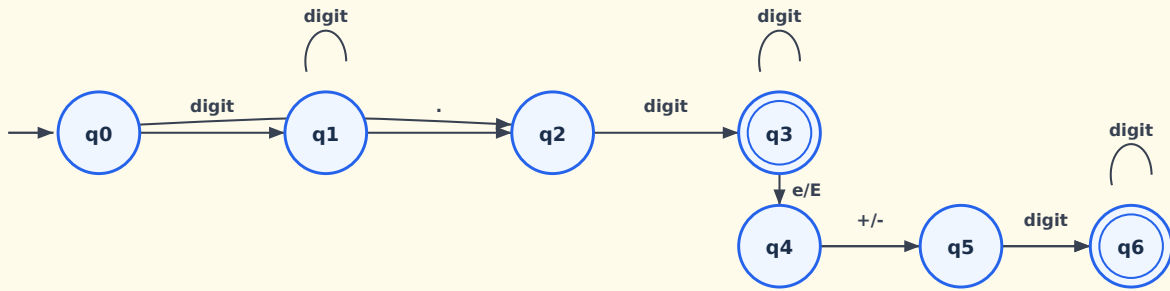
EN: Design a state diagram to recognize the floating-point literals of your favorite programming language.

TR: Favori programlama dilinizin kayan noktalı sayı sabitlerini (floating-point literals) tanıyan bir durum diyagramı tasarlayın.

● Cevap / Yönlendirme

C/Java kayan noktalı sabitler için durum diyagramı. Kabul eden durumlar: q_3 (ondalık kısım) ve q_6 (üs kısmı). Örnek geçerli sayılar: 3.14, .5, 2.0e10, 1.5E-3. Durum diyagramı aşağıda gösterilmiştir.

○ Durum Diyagramı — Kayan Noktalı Sayı Sabitleri (C/Java)



q0	Başlangıç durumu
q1	Tamsayı kısmı (rakamlar)
q2	. görüldü — kesirli kısım bekleniyor
q3	Kesirli rakamlar [KABUL]
q4	e/E görüldü — üs bekleniyor
q5	+/- işareti görüldü
q6	Üs rakamları [KABUL]

Alıştırma 3

EN: Write and test the code to implement the state diagram of Problem 1 (C-style /* */ comments).

TR: Problem 1'deki durum diyagramını (C tarzı /* */ yorumlar) uygulayan kodu yazın ve test edin.

● Cevap / Yönlendirme

Python implementasyonu:

```

def is_c_comment(s):
    state = 0
    for ch in s:
        if state==0: state=1 if ch=='/' else 0
        elif state==1: state=2 if ch=='*' else 0
        elif state==2: state=3 if ch=='*' else 2
        elif state==3:
            if ch=='/': return True
            elif ch=='*': state=3
            else: state=2
    return False
Test: is_c_comment('/* yorum */') → True
is_c_comment('/* kapalı') → False
  
```

Alıştırma 4

EN: Write and test the code to implement the state diagram of Problem 2 (floating-point literals).

TR: Problem 2'deki durum diyagramını (kayan noktalı sayılar) uygulayan kodu yazın ve test edin.

● Cevap / Yönlendirme

Python implementasyonu:

```
import re
def is_float(s):
    pattern = r'^[+-]?(\d+\.\d*|\.\d+)([eE][+-]?\d+)?$'
    return bool(re.match(pattern, s))
```

Alternatif durum makinesi uygulaması: Her karakter okunur, state geçişleri yapılır. Kabul eden durumlar q3 ve q6'dır.

Test: `is_float('3.14') → True`; `is_float('1.5e-3') → True`; `is_float('abc') → False`

Alıştırma 5

EN: Modify the lexical analyzer in Section 4.2 to recognize: `for(30)`, `if(31)`, `else(32)`, `while(33)`, `do(34)`, `int(35)`, `float(36)`, `switch(37)`.

TR: Bölüm 4.2'deki sözdizim analizörünü aşağıdaki ayrılmış kelimeleri tanıyacak biçimde değiştirin: `for(30)`, `if(31)`, `else(32)`, `while(33)`, `do(34)`, `int(35)`, `float(36)`, `switch(37)`.

● Cevap / Yönlendirme

Sözlüksel analizörün güncellemesi için ayrılmış kelime tablosu (lookup table) eklenir:

```
const char* RESERVED[] = {"for", "if", "else", "while", "do", "int", "float", "switch"};
```

```
const int CODES[] = { 30, 31, 32, 33, 34, 35, 36, 37};
```

Bir identifier tanındığında, bu tablo linear veya hash aramasıyla kontrol edilir:

```
int lookup(char* word) {
    for (int i=0; i<8; i++)
        if (strcmp(word, RESERVED[i])==0) return CODES[i];
    return IDENT_CODE; // ayrılmış kelime değilse tanımlayıcı
}
```

Alıştırma 6

EN: Convert the lexical analyzer (written in C) given in Section 4.2 to Java.

TR: Bölüm 4.2'deki C ile yazılmış sözlüksel analizörü Java'ya dönüştürün.

● Cevap / Yönlendirme

C'den Java'ya dönüşümde temel değişiklikler:

- char dizileri → String ve charAt()
- Global değişkenler → sınıf üye değişkenleri (fields)
- printf/scanf → System.out.print / Scanner
- Pointer aritmetiği → dizi indeksleme
- nextChar() fonksiyonu → reader.read() ile uygulanır

Java sınıf yapısı:

```
public class LexicalAnalyzer {  
    private char nextChar;  
    private String lexeme;  
    private int nextToken;  
    public void lex() { /* token okuma mantığı */ }  
}
```

Alıştırma 7

EN: Convert the recursive descent parser routines for <expr>, <term>, and <factor> given in Section 4.4.1 to Java.

TR: Bölüm 4.4.1'deki <expr>, <term> ve <factor> için özyinelemeli iniş ayrıştırıcı rutinlerini Java'ya dönüştürün.

● **Cevap / Yönlendirme**

Java dönüşümü:

```
public class RecursiveDescentParser {
    private int nextToken;
    private void match(int token) {
        if (nextToken==token) lex();
        else error();
    }
    private void expr() {
        term();
        while (nextToken==PLUS_CODE || nextToken==MINUS_CODE) {
            match(nextToken); term();
        }
    }
    private void term() {
        factor();
        while (nextToken==MULT_CODE || nextToken==DIV_CODE) {
            match(nextToken); factor();
        }
    }
    private void factor() {
        if (nextToken==IDENT_CODE) match(IDENT_CODE);
        else if (nextToken==LP_CODE) { match(LP_CODE); expr(); match(RP_CODE); }
        else error();
    }
}
```

Alıştırma 8

EN: For those rules that pass the pairwise disjointness test in Problem 1, write a recursive-descent parsing subprogram.

TR: Problem 1'deki ikili ayrışıklık testini geçen kurallar için özyinelemeli iniş ayrıştırma alt programı yazın.

● **Cevap / Yönlendirme**

Problem 1'den geçen kurallar: (a) $A \rightarrow aB \mid b \mid cBB$ ve (c) $C \rightarrow aaA \mid b \mid caB$

```
void parseA() {
  if (nextToken == 'a') { match('a'); parseB(); }
  else if (nextToken == 'b') { match('b'); }
  else if (nextToken == 'c') { match('c'); parseB(); parseB(); }
  else error();
}

void parseC() {
  if (nextToken == 'a') { match('a'); match('a'); parseA(); }
  else if (nextToken == 'b') { match('b'); }
  else if (nextToken == 'c') { match('c'); match('a'); parseB(); }
  else error();
}
```

Alıştırma 9

EN: For those rules that pass the pairwise disjointness test in Problem 2, write a recursive-descent parsing subprogram.

TR: Problem 2'deki ikili ayrışıklık testini geçen kurallar için özyinelemeli iniş ayrıştırma alt programı yazın.

● Cevap / Yönlendirme

Problem 2'den geçen kurallar: (a) $S \rightarrow aSb \mid bAA$ ve (b) $A \rightarrow b\{aB\} \mid a$

```
void parseS() {
  if (nextToken=='a') { match('a'); parseS(); match('b'); }
  else if (nextToken=='b') { match('b'); parseA(); parseA(); }
  else error();
}

void parseA() {
  if (nextToken=='b') {
    match('b');
    while (nextToken=='a') { match('a'); parseB(); } // {aB}
  } else if (nextToken=='a') { match('a'); }
  else error();
}
```

Alıştırma 10

EN: Implement and test the LR parsing algorithm given in Section 4.5.3.

TR: Bölüm 4.5.3'teki LR ayrıştırma algoritmasını uygulayın ve test edin.

● Cevap / Yönlendirme

LR ayrıştırıcı implementasyonu için temel bileşenler:

1. ACTION ve GOTO tabloları: 2D dizi veya HashMap

2. Ana döngü (Python tarzı pseudocode):

```
def lr_parse(tokens):
    stack = [0] # durum yığını
    tokens.append('$') # EOF
    i = 0
    while True:
        s = stack[-1]; a = tokens[i]
        act = ACTION[s][a]
        if act == 'accept': return True
        elif act.startswith('s'): # shift
            stack.append(int(act[1:])); i+=1
        elif act.startswith('r'): # reduce
            rule = int(act[1:])
            for _ in range(rule_length[rule]): stack.pop()
            stack.append(GOTO[stack[-1]][rule_lhs[rule]])
        else: return False # hata
```

Alıştırma 11

EN: Write an EBNF rule that describes the while statement of Java or C++. Write the recursive-descent subprogram for this rule.

TR: Java veya C++'ın while deyimini açıklayan EBNF kuralı yazın ve bu kural için özyinelemeli iniş alt programı yazın.

● Cevap / Yönlendirme

EBNF:

`<while_stmt> → while ( <expr> ) <stmt>`

`<stmt> → <block> | <simple_stmt>`

`<block> → { {<stmt>} }`

Java/C++ Recursive Descent:

```
void parseWhile() {
    match(WHILE_CODE);
    match(LEFT_PAREN);
    parseExpr();
    match(RIGHT_PAREN);
    parseStmt();
}

void parseStmt() {
    if (nextToken == LEFT_BRACE) parseBlock();
    else parseSimpleStmt();
}

void parseBlock() {
    match(LEFT_BRACE);
    while (nextToken != RIGHT_BRACE) parseStmt();
    match(RIGHT_BRACE);
}
```

Alıştırma 12

EN: Write an EBNF rule that describes the for statement of Java or C++. Write the recursive-descent subprogram for this rule.

TR: Java veya C++'ın for deyimini açıklayan EBNF kuralı yazın ve bu kural için özyinelemeli iniş alt programı yazın.

● Cevap / Yönlendirme

EBNF:

```
<for_stmt> → for ( [<init>] ; [<expr>] ; [<update>] ) <stmt>  
<init> → <type> <id> = <expr> | <expr>  
<update> → <expr> {, <expr>}
```

Java/C++ Recursive Descent:

```
void parseFor() {  
    match(FOR_CODE);  
    match(LEFT_PAREN);  
    if (nextToken != SEMICOLON) parseInit();  
    match(SEMICOLON);  
    if (nextToken != SEMICOLON) parseExpr();  
    match(SEMICOLON);  
    if (nextToken != RIGHT_PAREN) parseUpdate();  
    match(RIGHT_PAREN);  
    parseStmt();  
}
```